

Laboratory Work #2

Normalization & DDL

Normalization

Task 1

The **Booking_receipt** table violates the First Normal Form (1NF).

A relation is in 1NF if every value it stores is single-valued and atomic. The **seat_numbers** column stores a comma-separated list of values (e.g. "12A, 12B"), which means the values are not atomic — this directly violates the 1NF constraint.

Task 2

1NF

To bring the relation to 1NF, we remove the repeating group of values in **seat_numbers** by splitting the list of seats into separate rows. After the split, one row corresponds to one passenger within one booking, so the primary key becomes composite: (**booking_id**, **passenger_passport_number**).

Booking_receipt 1NF	
<u>booking_id</u> (PK)	
<u>passenger_passport_number</u> (PK)	
passenger_full_name	
flight_number	
departure_airport_name	
departure_city	
arrival_airport_name	
arrival_city	
airline_name	
seat_number	
ticket_price	

2NF

A relation is in 2NF if it satisfies 1NF and every non-key attribute is fully (not partially) dependent on the whole composite primary key. Attributes that depend on only part of the key are moved into separate tables.

Passengers	
<u>passenger_passport_number</u> (PK)	
passenger_full_name	

Flights 2NF	
<u>flight_number</u> (PK)	
departure_airport_name	
departure_city	
arrival_airport_name	
arrival_city	
airline_name	

Bookings
<u>booking_id</u> (PK)
flight_number (FK)
ticket_price

Tickets
<u>booking_id</u> (PK, FK)
<u>passenger passport number</u> (PK, FK)
seat_number

3NF

A relation is in 3NF if it satisfies all 2NF conditions and every transitive dependency of non-key attributes on the key has been removed. Cities depend on airports (not directly on the flight), so we move them into a separate table.

Airports
<u>airport_name</u> (PK)
city

Flights_3NF
<u>flight_number</u> (PK)
departure_airport_name (FK)
arrival_airport_name (FK)
airline_name

The Passengers, Bookings and Tickets tables remain the same as in 2NF.

Task 3

Functional dependencies used to justify each normalization step in Task 2:

- booking_id, passenger_passport_number → seat_number
- booking_id → flight_number, ticket_price
- passenger_passport_number → passenger_full_name
- flight_number → departure_airport_name, departure_city, arrival_airport_name, arrival_city, airline_name
- departure_airport_name → departure_city (transitive dependency)
- arrival_airport_name → arrival_city (transitive dependency)

Task 4

Current Primary Key: flight_id

Functional Dependencies:

flight_id → sch_departure_time, sch_arrival_time, departing_airport_id, arriving_airport_id, departing_gate, arriving_gate, airline_id, act_departure_time, act_arrival_time, created_at, updated_at

3NF Check: Yes, the Flights table is in 3NF.

A relation in 3NF must have no transitive dependencies of non-key attributes on the key column. Since airport and airline data has been moved into separate tables (Airport, Airline) and is represented here only by foreign keys (departing_airport_id, arriving_airport_id, airline_id), there are no transitive dependencies within the Flights table.

Task 5

SQL Implementation

SQL Implementation (PostgreSQL)

```
-- 1. Create tables
CREATE TABLE Airports (
  airport_name VARCHAR(100) PRIMARY KEY,
  city VARCHAR(100) NOT NULL
);
CREATE TABLE Passengers (
  passenger_passport_number VARCHAR(20) PRIMARY KEY,
  passenger_full_name VARCHAR(100) NOT NULL
);
CREATE TABLE Flights_3NF (
  flight_number VARCHAR(20) PRIMARY KEY,
  departure_airport_name VARCHAR(100) REFERENCES Airports(airport_name),
  arrival_airport_name VARCHAR(100) REFERENCES Airports(airport_name),
  airline_name VARCHAR(100) NOT NULL
);
CREATE TABLE Bookings (
  booking_id INT PRIMARY KEY,
  flight_number VARCHAR(20) REFERENCES Flights_3NF(flight_number),
  ticket_price DECIMAL(7, 2) NOT NULL
);
CREATE TABLE Tickets (
  booking_id INT REFERENCES Bookings(booking_id),
  passenger_passport_number VARCHAR(20) REFERENCES Passengers(passenger_passport_number),
  seat_number VARCHAR(5) NOT NULL,
  PRIMARY KEY (booking_id, passenger_passport_number)
);

-- 2. Insert data into tables
INSERT INTO Airports (airport_name, city) VALUES
  ('JFK', 'New York'),
  ('LHR', 'London'),
  ('ALA', 'Almaty');

INSERT INTO Passengers (passenger_passport_number, passenger_full_name) VALUES
  ('N123456', 'John Doe'),
  ('N654321', 'Jane Smith'),
  ('K298765', 'Aliya Kyzy');

INSERT INTO Flights_3NF (flight_number, departure_airport_name, arrival_airport_name, airline_name) VALUES
  ('BA117', 'LHR', 'JFK', 'British Airways'),
  ('KC909', 'ALA', 'LHR', 'Air Astana');

INSERT INTO Bookings (booking_id, flight_number, ticket_price) VALUES
  (1001, 'BA117', 550.00),
  (1002, 'BA117', 550.00),
  (1003, 'K298765', 420.50);

INSERT INTO Tickets (booking_id, passenger_passport_number, seat_number) VALUES
  (1001, 'N123456', '12A'),
  (1002, 'N654321', '12B'),
  (1003, 'K298765', '4C');
```

Object Explorer Servers (1) PostgreSQL 18 Databases (1) postgres Casts Catalogs Event Triggers Extensions Foreign Data Wrappers Languages Publications Schemas (1) Subscriptions Login/Group Roles Tablespaces

postgres/postgres@PostgreSQL 18

Query Query History

```
1 SELECT * FROM Airports;
```

Scratch Pad

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	airport_name [PK] character varying (100)	city character varying (100)
1	JFK	New York
2	LHR	London
3	ALA	Almaty

Servers > PostgreSQL 18 > Databases > postgres
default administrative connection database

Showing rows: 3 Query complete 00:00:00.134 LF Ln 1, Col 24

Object Explorer Servers (1) PostgreSQL 18 Databases (1) postgres Casts Catalogs Event Triggers Extensions Foreign Data Wrappers Languages Publications Schemas (1) Subscriptions Login/Group Roles Tablespaces

postgres/postgres@PostgreSQL 18

Query Query History

```
1 SELECT * FROM Airports;  
2 SELECT * FROM Passengers;
```

Scratch Pad

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

	passenger_passport_number [PK] character varying (20)	passenger_full_name character varying (100)
1	N123456	John Doe
2	N654321	Jane Smith
3	KZ98765	Aliya Kzyz

Servers > PostgreSQL 18 > Databases > postgres
default administrative connection database

Showing rows: 3 Query complete 00:00:00.074 LF Ln 2, Col 25

Object Explorer Servers (1) PostgreSQL 18 Databases (1) postgres Casts Catalogs Event Triggers Extensions Foreign Data Wrappers Languages Publications Schemas (1) Subscriptions Login/Group Roles Tablespaces

postgres/postgres@PostgreSQL 18

Query Query History

```
1 SELECT * FROM Airports;
2 SELECT * FROM Passengers;
3 SELECT * FROM Flights_3NF;
4 SELECT * FROM bookings_n;
5 SELECT * FROM Tickets;
6
7
```

Data Output Messages Notifications

Showing rows: 1 to 2 Page No: 1 of 1

flight_number	departure_airport_name	arrival_airport_name	airline_name
BA117	LHR	JFK	British Airways
KC909	ALA	LHR	Air Astana

Successfully run. Total query runtime: 111 msec. 2 rows affected.

Object Explorer Servers (1) PostgreSQL 18 Databases (1) postgres Casts Catalogs Event Triggers Extensions Foreign Data Wrappers Languages Publications Schemas (1) Subscriptions Login/Group Roles Tablespaces

postgres/postgres@PostgreSQL 18

Query Query History

```
1 SELECT * FROM Airports;
2 SELECT * FROM Passengers;
3 SELECT * FROM Flights_3NF;
4 SELECT * FROM Bookings;
5 SELECT * FROM Tickets;
6
7
```

Data Output Messages Notifications

Showing rows: 1 to 3 Page No: 1 of 1

booking_id	flight_number	ticket_price
1001	BA117	550.00
1002	BA117	550.00
1003	KC909	420.50

Successfully run. Total query runtime: 64 msec. 3 rows affected.

Object Explorer | Servers (1) | PostgreSQL 18 | Databases (1) | postgres

Query | Query History | Scratch Pad

```
1 CREATE TABLE Airports (  
2     airport_name VARCHAR(100) PRIMARY KEY,  
3     city VARCHAR(100) NOT NULL  
4 );  
5  
6 CREATE TABLE Passengers (  
7     passenger_passport_number VARCHAR(20) PRIMARY KEY,  
8     passenger_full_name VARCHAR(100) NOT NULL  
9 );  
10  
11 CREATE TABLE Flights_3NF (  
12     flight_number VARCHAR(20) PRIMARY KEY,  
13     departure_airport_name VARCHAR(100) REFERENCES Airports(airport_name),  
14     arrival_airport_name VARCHAR(100) REFERENCES Airports(airport_name),  
15     airline_name VARCHAR(100) NOT NULL  
16 );  
17  
18 CREATE TABLE Bookings (  
19     booking_id INT PRIMARY KEY,  
20     flight_number VARCHAR(20) REFERENCES Flights_3NF(flight_number),  
21     ticket_price DECIMAL(7, 2) NOT NULL  
22 );  
23  
24 CREATE TABLE Tickets (  
25     booking_id INT REFERENCES Bookings(booking_id),  
26     passenger_passport_number VARCHAR(20) REFERENCES Passengers(passenger_passport_number),  
27     seat_number VARCHAR(5) NOT NULL,  
28     PRIMARY KEY (booking_id, passenger_passport_number)  
29 );  
30  
31 INSERT INTO Airports (airport name, city) VALUES
```

Data Output | Messages | Notifications

ERROR: relation "airports" already exists

SQL state: 42P07

Discord

Object Explorer | Servers (1) | PostgreSQL 18 | Databases (1) | postgres

Query | Query History | Scratch Pad

```
1 SELECT * FROM Airports;  
2 SELECT * FROM Passengers;  
3 SELECT * FROM Flights_3NF;  
4 SELECT * FROM Bookings;  
5 SELECT * FROM Tickets;  
6  
7
```

Data Output | Messages | Notifications

Showing rows: 1 to 3 | Page No: 1 of 1

	booking_id [PK] integer	passenger_passport_number [PK] character varying (20)	seat_number character varying (5)
1	1001	N123456	12A
2	1002	N654321	12B
3	1003	KZ98765	4C

Total rows: 3 | Query complete 00:00:00.068 | LF | Ln 5, Col 1

DDL

2. SQL Implementation and DDL

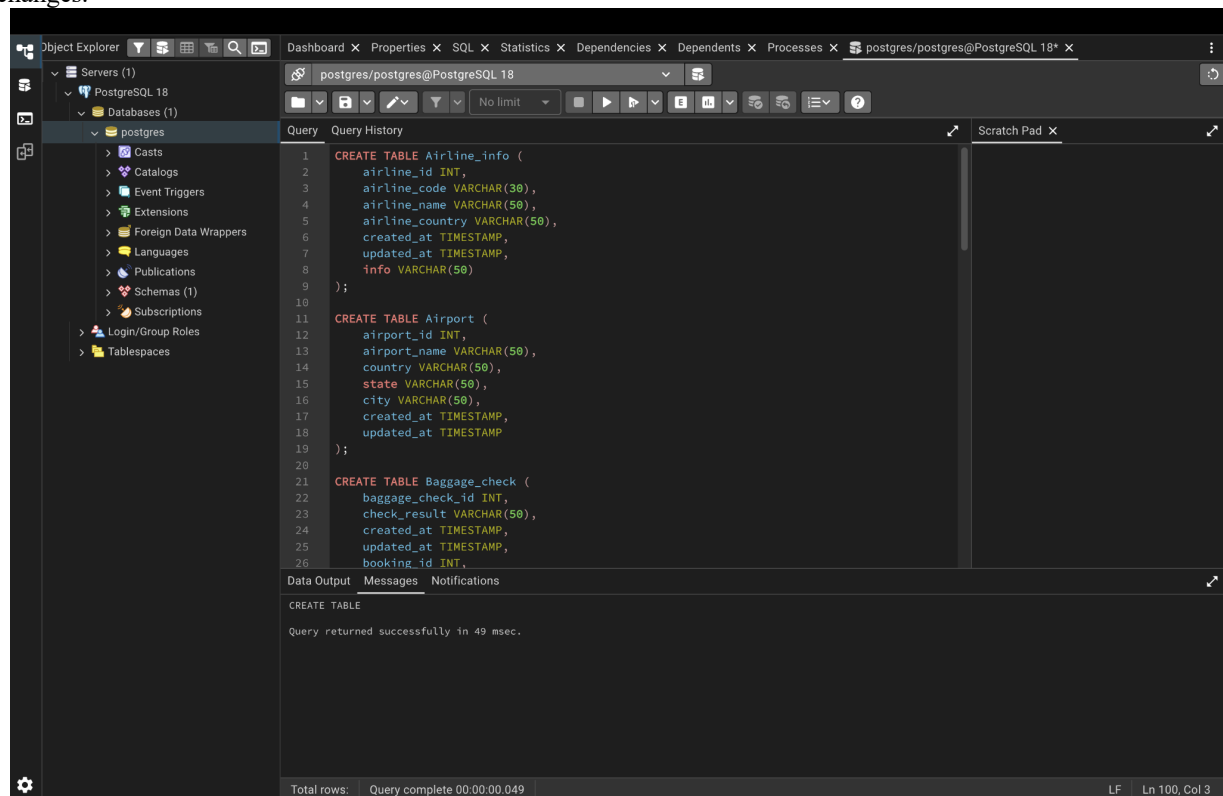
2.1 Task 5 — Database implementation

The normalized design was implemented in PostgreSQL. The DDL stage creates the required tables and then adds keys, constraints and relationships. The following ten tables are part of the implementation:

- Airline_info (renamed to Airline)
- Airport
- Baggage_check
- Baggage
- Boarding_pass
- Booking_flight
- Booking
- Flights
- Passengers
- Security_check

2.2 CREATE TABLE statements

The database tables were created with CREATE TABLE statements. The initial creation included the required attributes and data types; later ALTER TABLE statements were used to add constraints and apply the requested changes.



```
1 CREATE TABLE Airline_info (
2     airline_id INT,
3     airline_code VARCHAR(30),
4     airline_name VARCHAR(50),
5     airline_country VARCHAR(50),
6     created_at TIMESTAMP,
7     updated_at TIMESTAMP,
8     info VARCHAR(50)
9 );
10
11 CREATE TABLE Airport (
12     airport_id INT,
13     airport_name VARCHAR(50),
14     country VARCHAR(50),
15     state VARCHAR(50),
16     city VARCHAR(50),
17     created_at TIMESTAMP,
18     updated_at TIMESTAMP
19 );
20
21 CREATE TABLE Baggage_check (
22     baggage_check_id INT,
23     check_result VARCHAR(50),
24     created_at TIMESTAMP,
25     updated_at TIMESTAMP,
26     booking_id INT,
```

Query returned successfully in 49 msec.

Figure 1. CREATE TABLE statements executed successfully in pgAdmin.

2.3 Primary keys

Primary keys were added to uniquely identify records in each table:

```
ALTER TABLE Airline
ADD PRIMARY KEY (airline_id);

ALTER TABLE Airport
ADD PRIMARY KEY (airport_id);

ALTER TABLE Baggage_check
ADD PRIMARY KEY (baggage_check_id);

ALTER TABLE Baggage
ADD PRIMARY KEY (baggage_id);

ALTER TABLE Boarding_pass
ADD PRIMARY KEY (boarding_pass_id);

ALTER TABLE Booking_flight
ADD PRIMARY KEY (booking_flight_id);

ALTER TABLE Booking
ADD PRIMARY KEY (booking_id);

ALTER TABLE Flights
ADD PRIMARY KEY (flight_id);

ALTER TABLE Passengers
ADD PRIMARY KEY (passenger_id);

ALTER TABLE Security_check
ADD PRIMARY KEY (security_check_id);
```

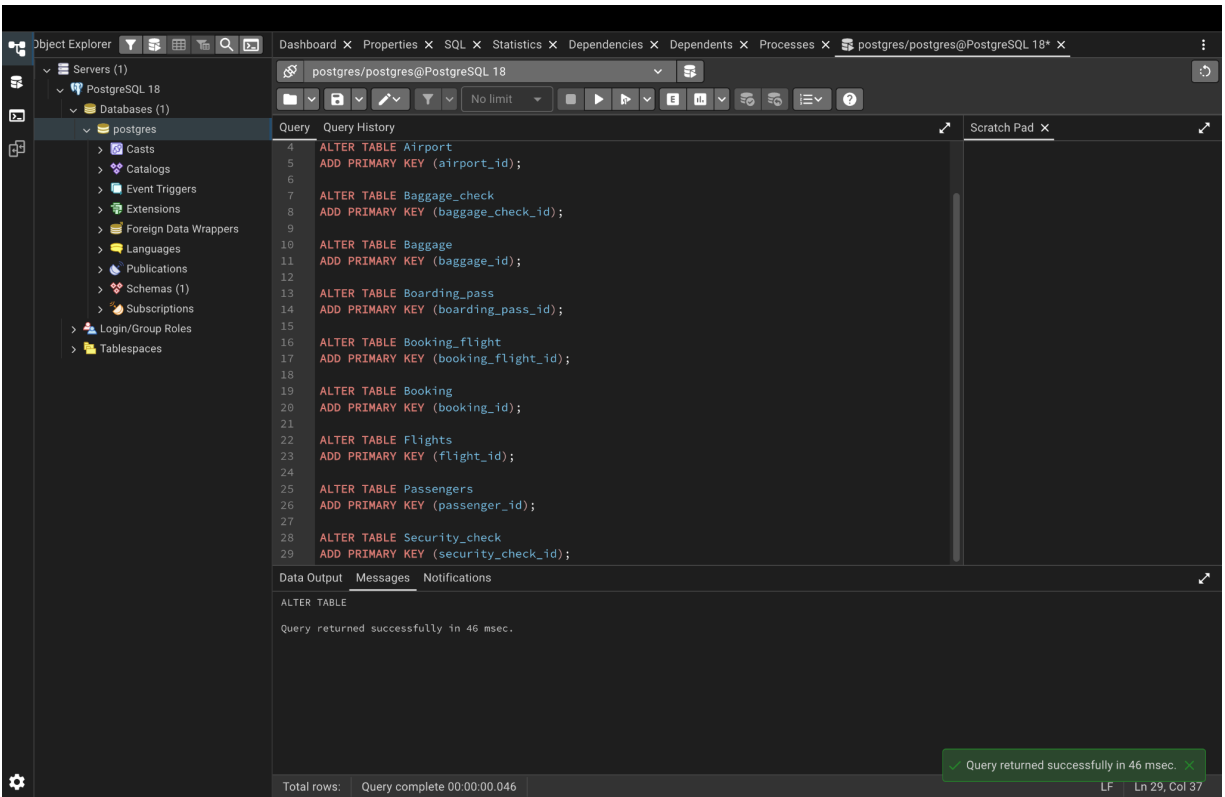


Figure 2. Primary-key definitions executed successfully.

2.4 NOT NULL constraints

NOT NULL constraints were applied to the required attributes. This prevents required fields from containing NULL values.

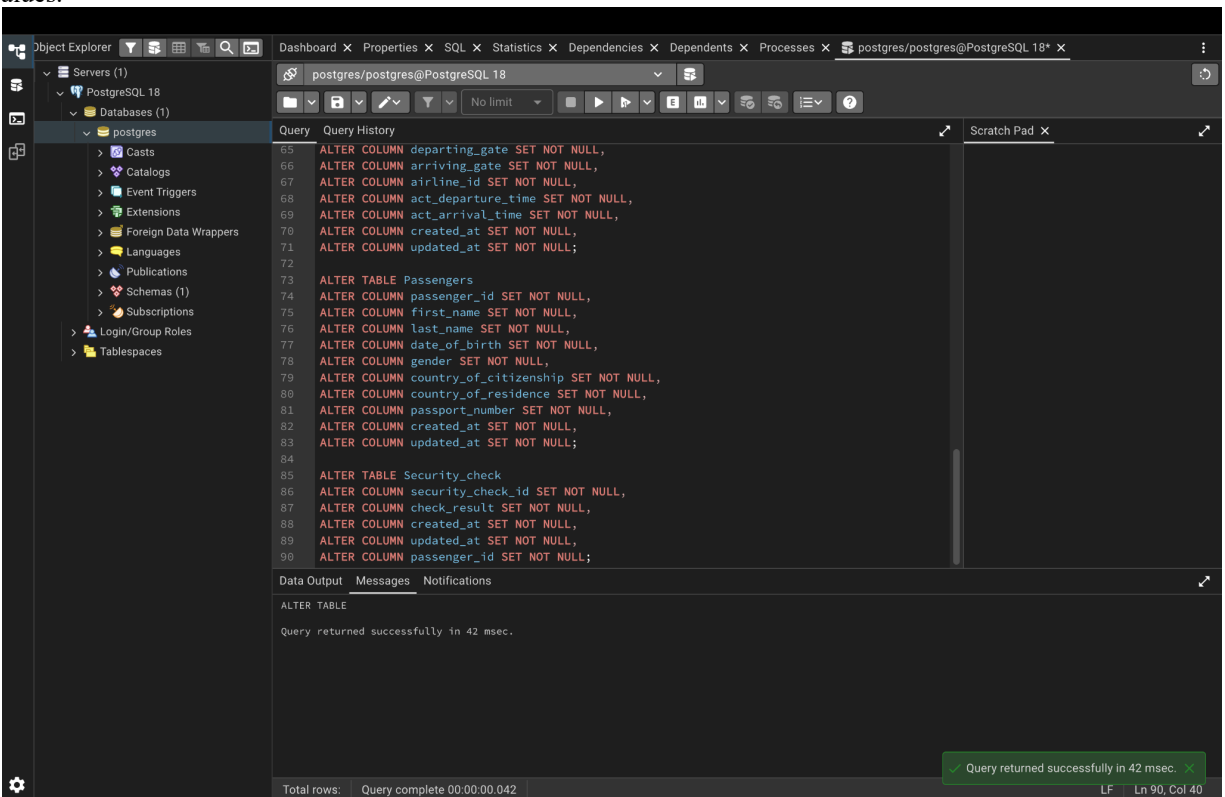


Figure 3. NOT NULL constraints applied to the database attributes.

2.5 ALTER TABLE operations

Rename Airline_info to Airline

```
ALTER TABLE Airline_info  
RENAME TO Airline;
```

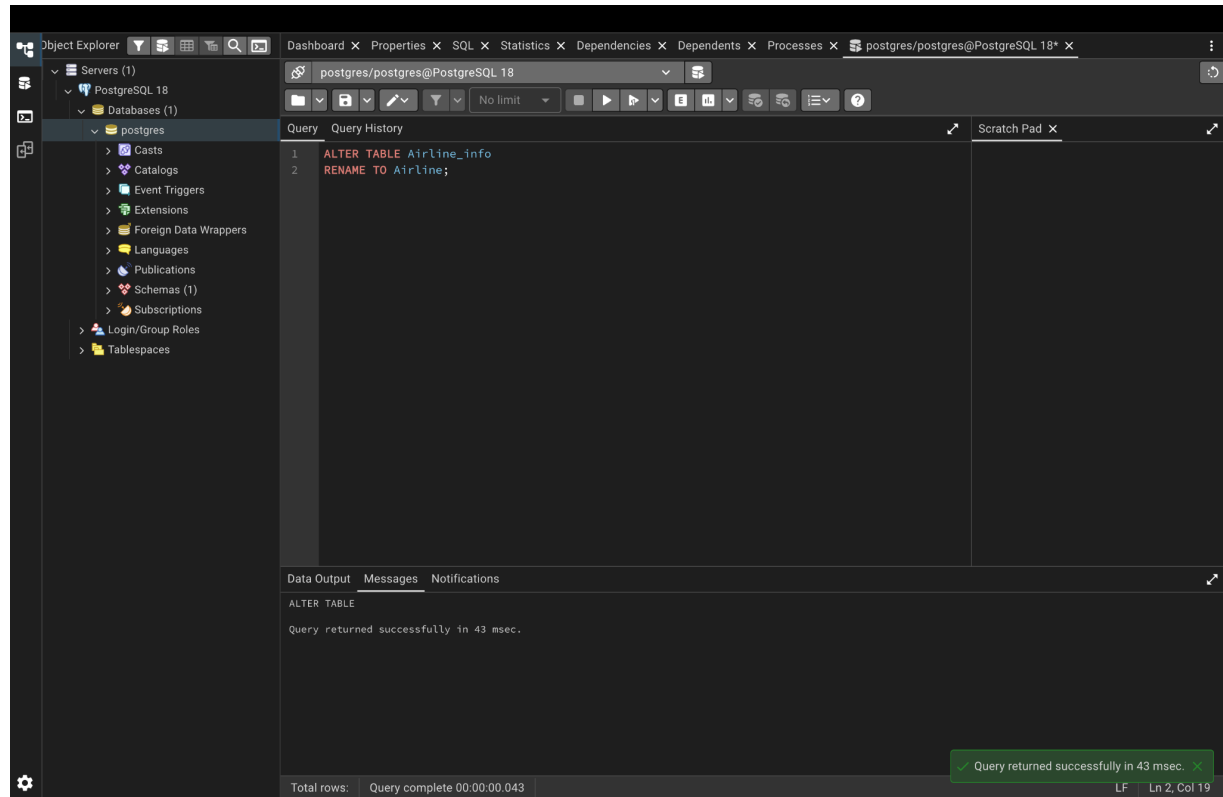


Figure 4. The Airline_info table was renamed to Airline.

Rename price to ticket_price

```
ALTER TABLE Booking  
RENAME COLUMN price TO ticket_price;
```

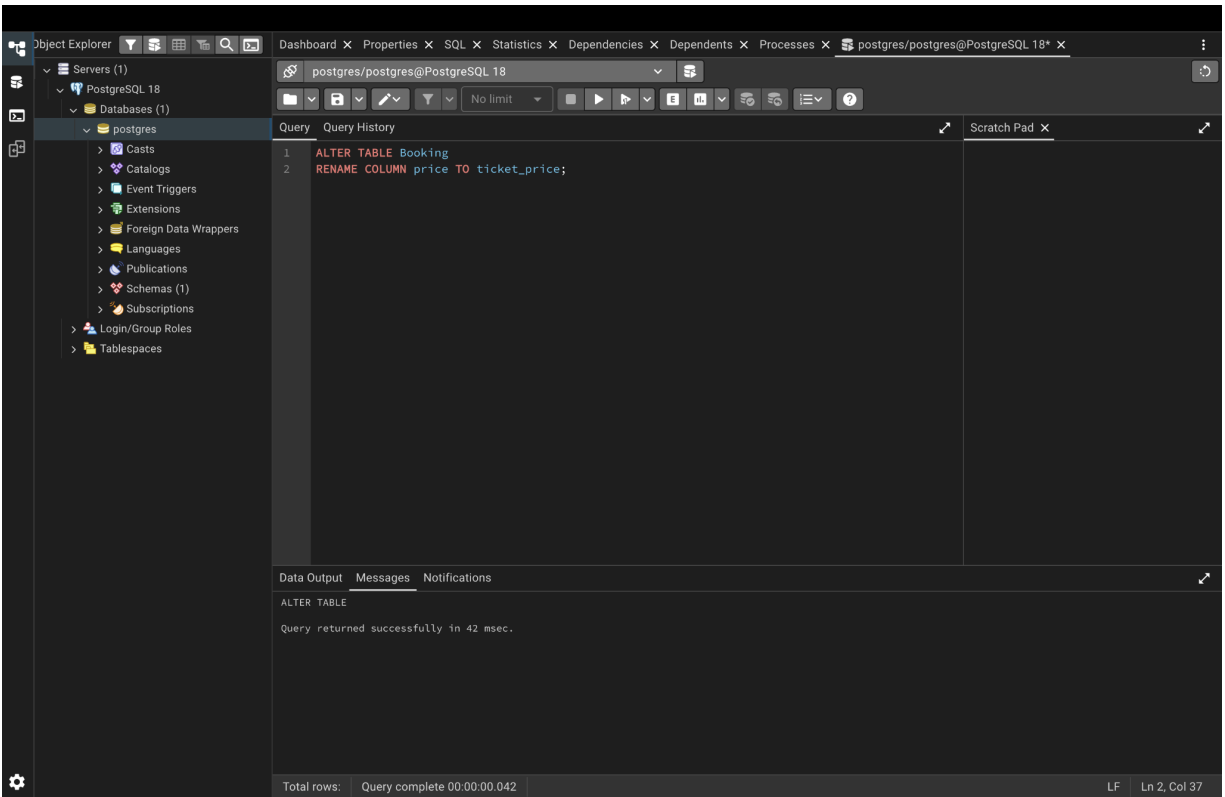


Figure 5. The price column was renamed to ticket_price.

Change departing_gate from VARCHAR(50) to TEXT

```
ALTER TABLE Flights
ALTER COLUMN departing_gate TYPE TEXT;
```

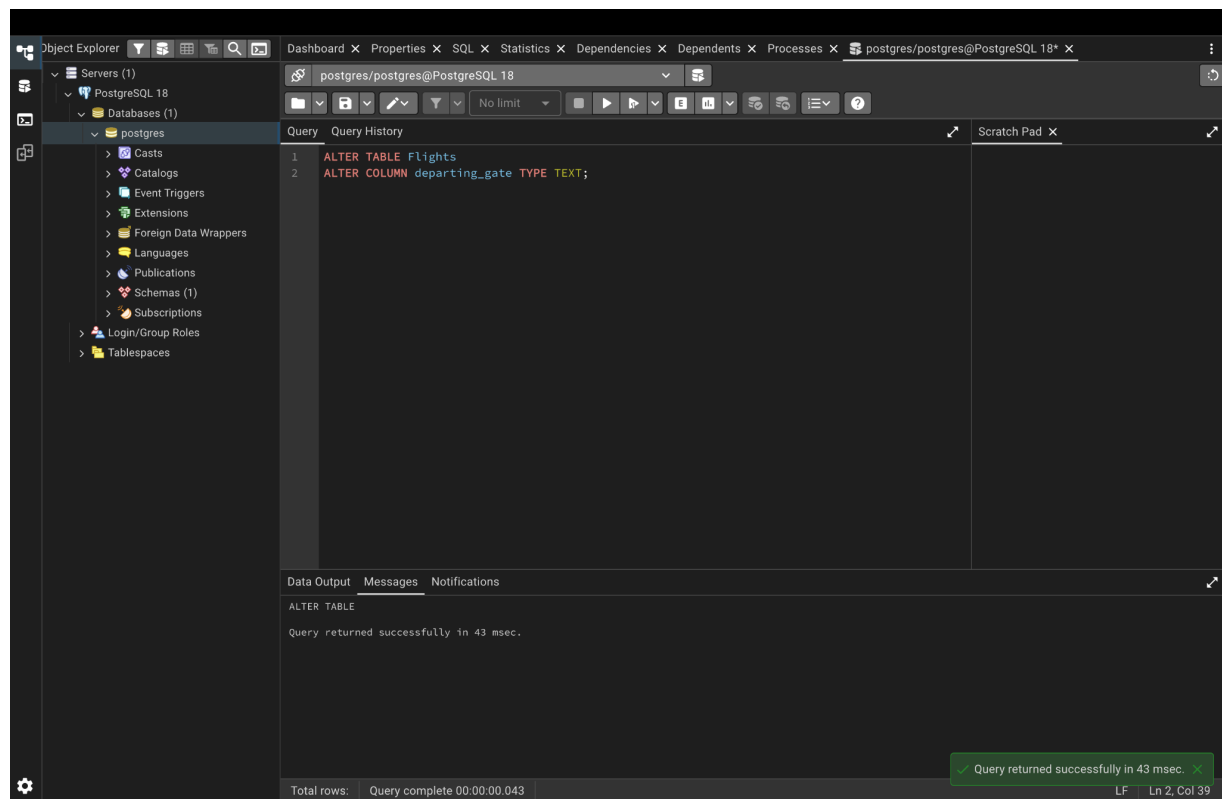


Figure 6. The `departing_gate` data type was changed to `TEXT`.

Drop the info column from Airline

```
ALTER TABLE Airline  
DROP COLUMN info;
```

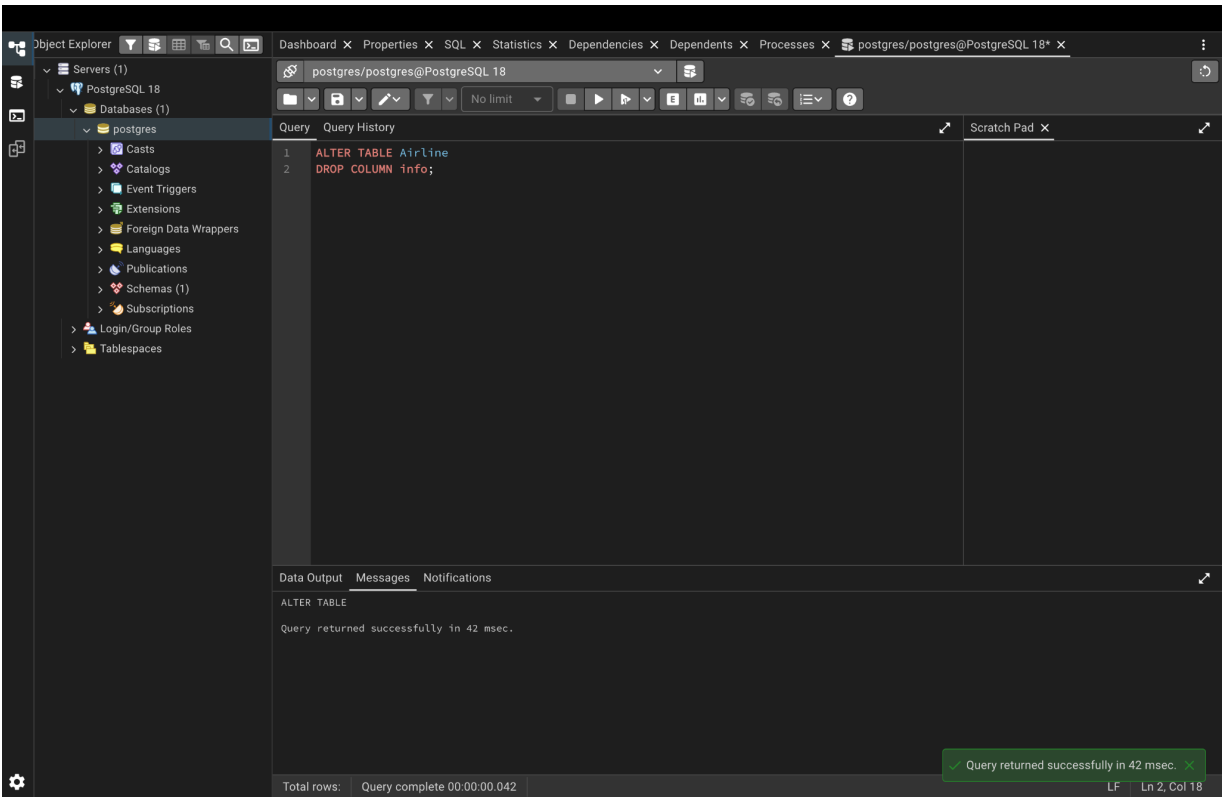


Figure 7. The unnecessary info column was removed from Airline.

2.6 Foreign-key relationships

The following foreign-key constraints establish the relationships between the tables:

```
ALTER TABLE Booking_flight
ADD CONSTRAINT fk_booking_flight_booking
FOREIGN KEY (booking_id)
REFERENCES Booking(booking_id);

ALTER TABLE Booking_flight
ADD CONSTRAINT fk_booking_flight_flight
FOREIGN KEY (flight_id)
REFERENCES Flights(flight_id);

ALTER TABLE Flights
ADD CONSTRAINT fk_flights_departing_airport
FOREIGN KEY (departing_airport_id)
REFERENCES Airport(airport_id);

ALTER TABLE Flights
ADD CONSTRAINT fk_flights_arriving_airport
FOREIGN KEY (arriving_airport_id)
REFERENCES Airport(airport_id);

ALTER TABLE Flights
```

```

ADD CONSTRAINT fk_flights_airline
FOREIGN KEY (airline_id)
REFERENCES Airline(airline_id);

```

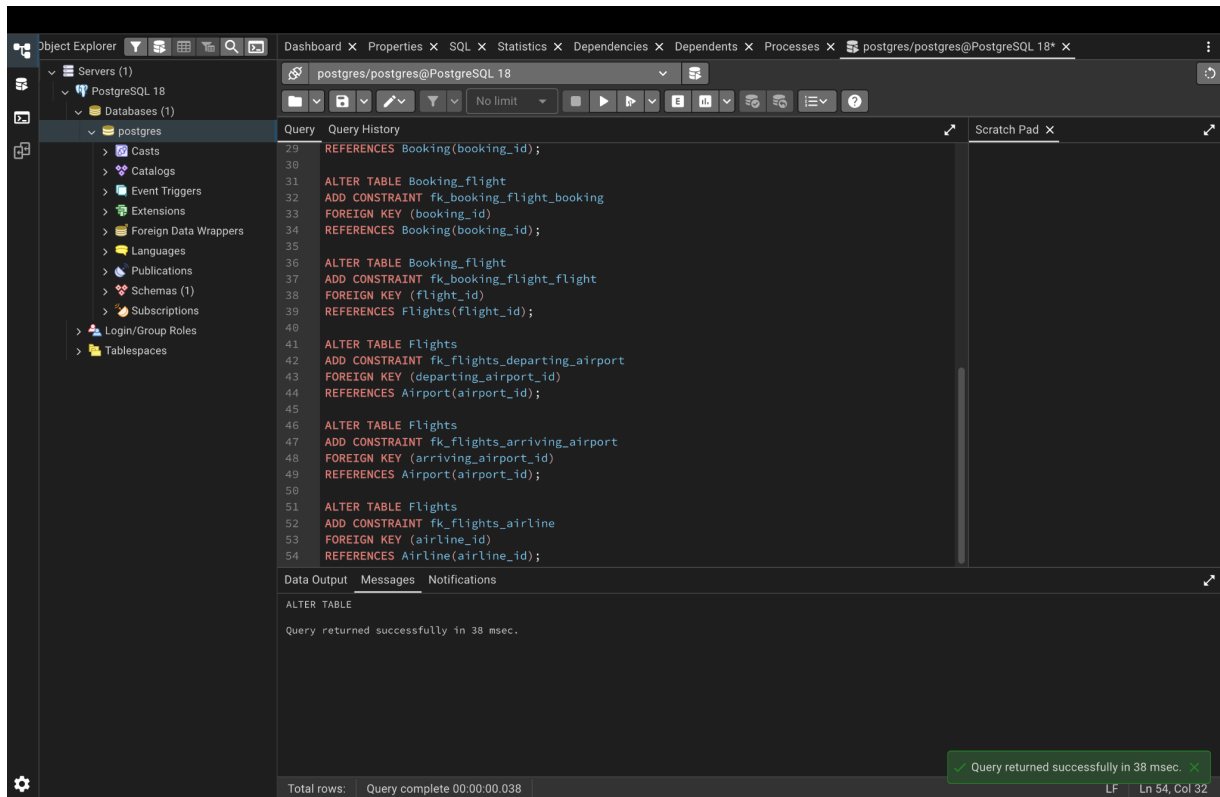


Figure 8. Foreign-key constraints were created successfully.

2.7 Verification query and result

After creating the relationships, the database metadata was queried through `information_schema` to verify which columns reference other tables.

```

SELECT
    tc.table_name,
    kcu.column_name,
    ccu.table_name AS referenced_table,
    ccu.column_name AS referenced_column
FROM information_schema.table_constraints AS tc
JOIN information_schema.key_column_usage AS kcu
    ON tc.constraint_name = kcu.constraint_name
    AND tc.table_schema = kcu.table_schema
JOIN information_schema.constraint_column_usage AS ccu
    ON ccu.constraint_name = tc.constraint_name
    AND ccu.table_schema = tc.table_schema
WHERE tc.constraint_type = 'FOREIGN KEY'
    AND tc.table_schema = 'public'
ORDER BY tc.table_name;

```

The screenshot shows a PostgreSQL IDE interface. On the left is the Object Explorer showing the database structure. The main window displays a SQL query and its results.

Query:

```

1  SELECT
2      tc.table_name,
3      kcu.column_name,
4      ccu.table_name AS referenced_table,
5      ccu.column_name AS referenced_column
6  FROM information_schema.table_constraints AS tc
7  JOIN information_schema.key_column_usage AS kcu
8      ON tc.constraint_name = kcu.constraint_name
9      AND tc.table_schema = kcu.table_schema

```

Data Output:

	table_name	column_name	referenced_table	referenced_column
1	baggage	booking_id	booking	booking_id
2	baggage_ch...	passenger_id	passengers	passenger_id
3	baggage_ch...	booking_id	booking	booking_id
4	boarding_pa...	booking_id	booking	booking_id
5	booking	passenger_id	passengers	passenger_id
6	booking_flight	flight_id	flights	flight_id
7	booking_flight	booking_id	booking	booking_id
8	flights	airline_id	airline	airline_id
9	flights	departing_airport...	airport	airport_id
10	flights	arriving_airport_id	airport	airport_id
11	security_che...	passenger_id	passengers	passenger_id

Total rows: 11 | Query complete 00:00:00.054

Figure 9. Verification query showing 11 foreign-key relationships.

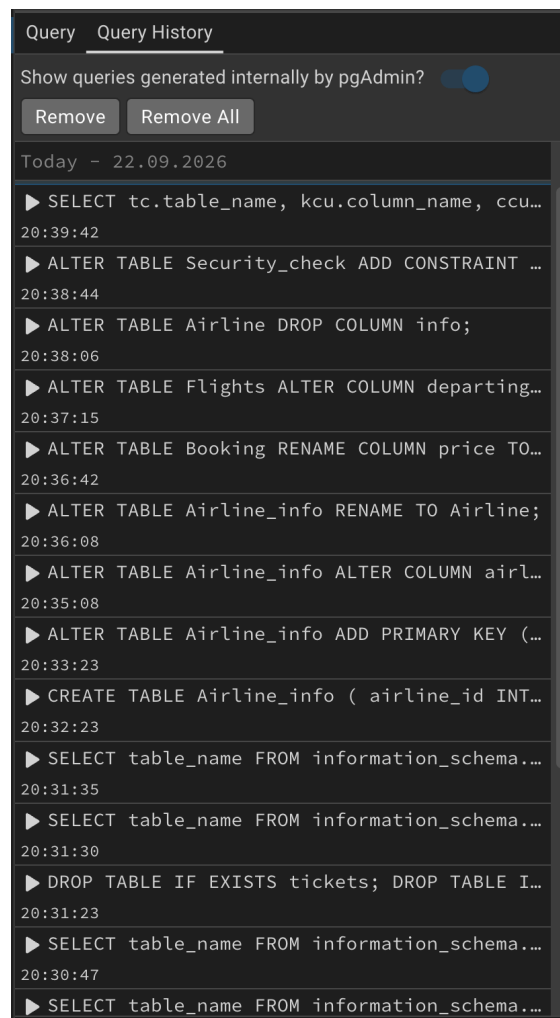
The verification result contains 11 foreign-key relationships. It confirms references from Baggage, Baggage_check, Boarding_pass, Booking, Booking_flight, Flights and Security_check to their related tables.

3. Conclusion

In this laboratory work, the database design was normalized from 1NF through 3NF by removing repeating groups, partial dependencies and transitive dependencies. Functional dependencies were used to explain why attributes were separated into Passengers, Flights, Bookings, Tickets and Airports.

The resulting design was then implemented in PostgreSQL using DDL. The required tables were created, primary keys and NOT NULL constraints were added, and several ALTER TABLE operations were performed to rename objects, change a data type and remove an unnecessary column. Foreign keys were then created to connect the tables.

Finally, the information_schema query confirmed the foreign-key relationships in the database. Therefore, the laboratory demonstrates both the theoretical normalization process and its practical implementation in PostgreSQL.



The screenshot shows the 'Query History' window in a PostgreSQL client. It displays a list of queries executed on '22.09.2026'. The queries include creating a table, selecting from information_schema, dropping a table, and several ALTER TABLE operations. Each query is preceded by a play button icon and followed by its execution time.

Query	Time
▶ SELECT table_name FROM information_schema...	20:30:47
▶ SELECT table_name FROM information_schema...	20:31:30
▶ DROP TABLE IF EXISTS tickets; DROP TABLE I...	20:31:23
▶ SELECT table_name FROM information_schema...	20:31:35
▶ CREATE TABLE Airline_info (airline_id INT...	20:32:23
▶ ALTER TABLE Airline_info ADD PRIMARY KEY (...	20:33:23
▶ ALTER TABLE Airline_info ALTER COLUMN airl...	20:35:08
▶ ALTER TABLE Airline_info RENAME TO Airline;	20:36:08
▶ ALTER TABLE Booking RENAME COLUMN price TO...	20:36:42
▶ ALTER TABLE Flights ALTER COLUMN departing...	20:37:15
▶ ALTER TABLE Airline DROP COLUMN info;	20:38:06
▶ ALTER TABLE Security_check ADD CONSTRAINT ...	20:38:44
▶ SELECT tc.table_name, kcu.column_name, ccu...	20:39:42